
Parsimonious Learning-Augmented Online Metric Matching

Yongho Shin^{* 1} Phanu Vajanopath^{* 1}

Abstract

Learning-augmented algorithms have received significant attention in recent years, particularly in the context of online optimization. Motivated by the high computational cost of generating predictions, a growing line of work studies the tradeoff between performance guarantees and the number of predictions used in learning-augmented algorithms for problems such as caching and metrical task systems. In this paper, we extend this line of research to *online metric matching* by developing *parsimonious* learning-augmented algorithms and establishing lower bounds on their performance. Our approach extends the Follow-the-Prediction framework to the parsimonious setting by filling in a *virtual prediction* in the absence of an actual prediction, using an online metric matching algorithm that maintains good intermediate matchings throughout its execution. We complement our theoretical results with an empirical evaluation, demonstrating the practical effectiveness of our approach.

1. Introduction

Online metric matching is a fundamental online optimization problem that naturally arises in applications such as ride-hailing and resource allocation (Kalyanasundaram & Pruhs, 1993; Khuller et al., 1994). The problem is defined as follows. We are given a metric containing n servers and n requests. The locations of the servers are known in advance, while the requests arrive one at a time step in an online fashion. Upon the arrival of a request, the algorithm must irrevocably match it to an available server at the moment; once matched, a server cannot be used again, and a request cannot be reassigned to another server at a later time step. At termination, the algorithm produces a perfect matching between servers and requests. The objective is to minimize

the cost of the perfect matching, measured as the sum of distances between matched pairs.

Due to a dedicated line of work (Kalyanasundaram & Pruhs, 1993; Khuller et al., 1994; Meyerson et al., 2006; Bansal et al., 2014), this problem is fairly well understood. On the algorithmic side, there exists a deterministic algorithm that outputs a perfect matching whose cost is at most a factor of $2n - 1$ larger than that of an optimal perfect matching with hindsight (Kalyanasundaram & Pruhs, 1993; Khuller et al., 1994). Later, Meyerson et al. (2006) gives a randomized algorithm returning a solution of expected cost within a factor of $O(\log^3 n)$ to the optimal cost with hindsight; this factor was further improved to $O(\log^2 n)$ by Bansal et al. (2014). It is also known that these guarantees are (nearly) best-possible. In particular, no deterministic online algorithm can achieve a factor better than $2n - 1$ (Kalyanasundaram & Pruhs, 1993; Khuller et al., 1994), matching the above algorithmic result. For randomized algorithms, an $\Omega(\log n)$ lower bound is known (Meyerson et al., 2006).

A major source of these strong lower bounds is the uncertainty about the future input. However, recent advances in AI and machine learning suggest that reasonable predictions about the future input may be available in many practical settings. Motivated by this observation, *learning-augmented algorithms*, also known as *algorithms with predictions*, have received significant attention in recent years, especially in online optimization since the seminal work of Lykouris & Vassilvitskii (2021). These algorithms are provided with predictions as part of the input and are designed to leverage them to solve the problem. The performance is typically measured by the following three factors (Kumar et al., 2018): *consistency*, which measures performance when predictions are accurate; *robustness*, which guarantees performance regardless of prediction quality; and *smoothness*, which captures how gracefully performance degrades as prediction error increases.

Online metric matching has also been studied in this learning-augmented framework. Antoniadis et al. (2023b) introduced the *action prediction* model and presented a deterministic algorithm that uses the prediction provided at every time and outputs a perfect matching with cost at most $9 \cdot \min\{\text{cost}(\text{OPT}) + 2\eta, (2n - 1) \text{cost}(\text{OPT})\}$,¹ where

¹This expression reflects the correct performance guarantee

^{*}Equal contribution ¹Institute of Computer Science, University of Wrocław, Wrocław, Poland. Correspondence to: Yongho Shin <yongho@cs.uni.wroc.pl>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

$\text{cost}(\text{OPT})$ denotes the cost of an optimal perfect matching with hindsight and η denotes the error of action prediction.

While predictions can substantially improve performance, generating them is often computationally expensive. This motivates the study of learning-augmented algorithms that use predictions parsimoniously. Im et al. (2022) initiated this line of work by studying the tradeoff between performance guarantee and a bounded number of predictions in online caching. Subsequently, Sadek & Eliás (2024) introduced a more restricted regime in which two predictions must be separated by a minimum time gap, and analyzed the tradeoffs for metrical task systems and caching. In this context, Im et al. (2022) posed a question of investigating a parsimonious prediction model for online metric matching.

Our Contributions We answer the question posed by Im et al. (2022) by presenting parsimonious learning-augmented algorithms for online metric matching under action predictions, together with lower bounds on their performance.

We first present our algorithmic results for general metric spaces. We denote by $\text{cost}(\text{OPT})$ the cost of an optimal perfect matching with hindsight, and by η the total error incurred by the actual predictions used by the algorithm.

Theorem 1.1. *For any $k \in \mathbb{N}$, there exists a deterministic learning-augmented algorithm for online metric matching that outputs a perfect matching with cost at most*

$$9 \cdot \min \left\{ \begin{array}{l} (2k - 1) \text{cost}(\text{OPT}) + 2k\eta, \\ (2n - 1) \text{cost}(\text{OPT}) \end{array} \right\},$$

where any two predictions used by the algorithm are separated by at least k time steps.

Theorem 1.2. *For any $k \in \mathbb{N}$, there exists a randomized learning-augmented algorithm for online metric matching that outputs a perfect matching with expected cost at most*

$$\min \left\{ \begin{array}{l} O(\log n \log k) \cdot (\text{cost}(\text{OPT}) + \eta), \\ O(\log^2 n) \cdot \text{cost}(\text{OPT}) \end{array} \right\},$$

where any two predictions used are deterministically separated by at least k time steps.

In addition, we obtain improved guarantees for specific metric spaces, including line metrics and hierarchically well-separated trees (see Section 3.3).

Our algorithms are built upon a meta-algorithm that can be viewed as a parsimonious variant of the Follow-the-Prediction framework of Antoniadis et al. (2023b). A key challenge is that this framework requires a prediction at

derivable from the analysis of Antoniadis et al. (2023b), which is inconsistent with the expression reported in the paper (Coester, 2026).

every time step, whereas in our algorithms, predictions can only be used sparingly with a minimum separation of k time steps between consecutive predictions.

To overcome this challenge, we construct *virtual predictions* to fill in between two consecutive actual predictions through an online metric matching algorithm that satisfies two properties, *adherence* and *strong competitiveness* (see Definitions 2.4 and 2.5). Intuitively, these properties quantify how well the algorithm maintains an intermediate matching throughout its execution. While such intermediate guarantees are not required in the classical setting—since the total number of requests is known upfront—many existing algorithms already satisfy these properties (see Table 1).

We leverage these algorithms $\mathcal{A}$ within our meta-algorithm to generate virtual predictions as follows. Denoting by P the previous actual prediction, we treat P as accurate, and we try to select additional servers from those unmatched in P to include in the virtual predictions. To this end, we define an auxiliary sub-instance in which the servers are those unmatched in P and the requests are those arriving between P and its subsequent actual prediction. We then run $\mathcal{A}$ on this sub-instance. At each time step, the servers matched by $\mathcal{A}$, together with P , form the virtual prediction for that time step. To guarantee a good quality of the constructed virtual predictions, it is desirable for $\mathcal{A}$ to maintain a high-quality matching throughout its execution, which is measured by adherence and strong competitiveness.

Next, we establish impossibility results by proving lower bounds on the cost incurred by any algorithm that uses predictions parsimoniously, even when the predictions are guaranteed to be accurate. These lower bounds are obtained by generalizing the standard hard instance on a star metric. It is noteworthy that our results show that the guarantee of Theorem 1.1 is essentially best-possible for deterministic algorithms up to a constant factor when the prediction is perfect, even in the more permissive setting where only the total number of predictions is bounded.

Finally, we empirically evaluate our theoretical results through experiments on instances that are either artificially synthesized or derived from real-world data. Our experimental results are broadly consistent with our expectations, demonstrating the practical effectiveness of our approach.

Further related work is discussed in Appendix A.

2. Preliminaries

Online Metric Matching In the *online metric matching* problem, we are given n servers S and n requests R contained in a metric (V, d) , i.e., for any $u, v, w \in V$, $d(u, u) = 0$, $d(u, v) = d(v, u)$, and $d(u, v) \leq d(u, w) + d(w, v)$. Two distinct servers or requests can be placed at the same loca-

tion in the metric; one can regard S and R as multisets of V . The servers are known from the beginning whilst the requests are given one at a round in an online manner. When a request $r \in R$ arrives, we need to assign r to a currently unmatched server $s \in S$; this decision is irrevocable, i.e., s is not allowed to be matched with any of subsequent requests, and r cannot be reassigned to any other server in later rounds. The objective is to find a perfect matching M between S and R at minimum total cost denoted by $\text{cost}(M) := \sum_{(s,r) \in M} d(s, r)$. For an online metric matching algorithm $\mathcal{A}$, we also write $\text{cost}(\mathcal{A})$ to denote the cost of matching output by $\mathcal{A}$.

Definition 2.1. For some $\rho : \mathbb{N} \rightarrow \mathbb{R}_+$, we say an algorithm $\mathcal{A}$ is ρ -competitive if $\text{cost}(\mathcal{A}) \leq \rho(n) \cdot \text{cost}(\text{OPT})$ for any instance, where OPT denotes a minimum-cost perfect matching between S and R in the metric with hindsight.

Throughout this paper, we assume $\text{cost}(\text{OPT}) \geq 1$. This assumption is without loss of generality by scaling the metric with an appropriate factor; if $\text{cost}(\text{OPT}) = 0$, one can easily construct an optimal perfect matching even in the online setting.

Learning Augmentation with Action Predictions For the learning-augmented setting of online metric matching, Antoniadis et al. (2023b) introduce the *action prediction* defined as a server set $P_t \subseteq S$ of size $|P_t| = t$ for every round $t \in \{1, \dots, n\}$.

The definition of prediction error requires a notion of *distance* between two “configurations” which is formally defined as follows: for any multisets A and B of V with $|A| = |B|$, we denote by $\text{dist}(A, B)$ the minimum cost of a perfect matching between A and B in the metric (V, d) . Here are some useful properties of this distance, whose proof is deferred to Appendix E.

Fact 2.2. We have the following:

1. For multisets $A, B, C \subseteq V$ with $|A| = |B|$, $\text{dist}(A, B) = \text{dist}(A \uplus C, B \uplus C)$.
2. For multisets $A, B, C \subseteq V$ with $|A| = |B| = |C|$, $\text{dist}(A, B) \leq \text{dist}(A, C) + \text{dist}(C, B)$.

We now define the error of action prediction. Fix an optimal perfect matching OPT with hindsight. For each $t \in \{1, \dots, n\}$, let $r_t \in R$ denote the request that arrives in round t , $R_t := \{r_1, \dots, r_t\}$ be the first t requests in their arrival order, and $O_t \subseteq S$ denote the set of servers matched by R_t in OPT . The prediction error η_t in round t is then defined to be $\eta_t := \text{dist}(P_t, O_t)$.

Parsimonious Learning Augmentation For the parsimonious learning-augmented setting (Im et al., 2022; Sadek

& Eliás, 2024), it is assumed that there exists an oracle providing a prediction to the algorithm in any round on request from the algorithm. We consider the following two parsimonious regimes studied by Sadek & Eliás (2024):

- *Well-separated queries to the oracle:* The algorithm receives a prediction once every k time steps, for some *separation parameter* k . Without loss of generality, the predictions are available to the algorithm in rounds ik for every $i \in \{1, \dots, \lfloor n/k \rfloor\}$.
- *Bounded number of predictions:* The algorithm is allowed to make a query to the oracle at any time under one hard constraint that the total number of queries must not exceed some budget $B \in \mathbb{N}$.

Note that the first regime is a special case of the second regime with budget $B = \lceil n/k \rceil - 1$.

The error of the oracle is defined to be the sum of errors of queried predictions, i.e., $\eta(Q) := \sum_{t \in Q} \eta_t$, where $Q \subseteq \{1, \dots, n\}$ denotes the rounds in which the algorithm makes queries to the oracle.

Combination Algorithm In what follows, we focus on presenting a learning-augmented algorithm that performs well when the prediction is good, but is not robust when the prediction is bad, because there is a way to make any algorithm robust at a constant factor in cost (Fiat et al., 1994; Antoniadis et al., 2023b). For the sake of completeness, we provide the formal proof in Appendix B.

Theorem 2.3. For two (possibly randomized) algorithms $\mathcal{A}$ and $\mathcal{B}$ for online metric matching, there exists a combination algorithm whose output incurs in expectation at most $9 \cdot \min\{\mathbb{E}[\text{cost}(\mathcal{A})], \mathbb{E}[\text{cost}(\mathcal{B})]\}$.

Adherence and Strong Competitiveness Let us further define a couple of algorithmic properties that will be helpful in analysis of our parsimonious learning-augmented algorithm. To define these properties, we fix an algorithm $\mathcal{A}$ and an instance (V, d, S, R) . For $t \in \{1, \dots, n\}$, let $\mathcal{M}_t$ denote the set of all matchings where exactly R_t is matched (i.e., all maximum matchings in the complete bipartite graph induced by $S \cup R_t$). Moreover, let $\mathcal{A}_t$ denote the matching maintained by $\mathcal{A}$ at the end of round t , and let S_t be the set of servers matched by $\mathcal{A}_t$.

Definition 2.4 (Adherence). For some $\gamma \geq 1$, we say an algorithm $\mathcal{A}$ is γ -adherent if, for every $t \in \{1, \dots, n\}$, $\text{dist}(S_t, R_t) \leq \gamma \cdot \min_{M \in \mathcal{M}_t} \{\text{cost}(M)\}$.

Definition 2.5 (Strong competitiveness). For some $\rho : \mathbb{N} \rightarrow \mathbb{R}$, we say an algorithm $\mathcal{A}$ is *strongly ρ -competitive* if, for every $t \in \{1, \dots, n\}$, $\mathbb{E}[\text{cost}(\mathcal{A}_t)] \leq \rho(t) \cdot \min_{M \in \mathcal{M}_t} \{\text{cost}(M)\}$.

We remark that $O(1)$ -adherence and strong competitiveness appear in many existing prediction-free algorithms. We provide in Table 1 a few such algorithms that we use to derive our parsimonious learning-augmented algorithms.

3. Algorithms

In this section, we present our parsimonious learning-augmented algorithms for online metric matching. They are built upon a meta-algorithm which is essentially the Follow-the-Prediction (FtP) algorithm of Antoniadis et al. (2023b) that queries the prediction to the oracle in every k round for some $k \in \mathbb{N}$ while generating *virtual predictions* for the other rounds using an adherent, strongly competitive algorithm. The main technical theorem about the meta-algorithm is as follows:

Theorem 3.1. *Given a γ -adherent, strongly ρ -competitive algorithm and $k \in \mathbb{N}$, there is an algorithm that outputs a perfect matching of (expected) cost at most*

$$(1 + \gamma + \rho(k-1)) \text{cost}(\text{OPT}) + (2 + \gamma + \rho(k-1)) \eta(Q),$$

where $Q = \{k, 2k, \dots, \lfloor n/k \rfloor \cdot k\}$ denotes the rounds in which the algorithm makes queries to the oracle.

Note that the meta-algorithm works for the regime of well-separated queries to the oracle with separation parameter k , and hence, it also works for the more general regime of bounded number of predictions with budget $B = \lfloor n/k \rfloor$.

We begin by reviewing the FtP algorithm in Section 3.1, and then describe the construction of (virtual) predictions and prove Theorem 3.1 in Section 3.2. Finally, in Section 3.3, we leverage existing algorithms within our meta-algorithm.

3.1. Follow-the-Prediction Algorithm

The FtP algorithm is described in Algorithm 1. Note that this algorithm requires a prediction P_t in every round $t \in \{1, \dots, n\}$.

The FtP algorithm is guaranteed to output a feasible perfect matching between S and R because $p_t \in P_t \setminus P_{t-1}$ due to μ_1 and $s_t \in S \setminus S_{t-1}$ due to μ_2 in every round t .

Following is the key lemma for competitiveness of the FtP algorithm. We include its proof in Appendix E for completeness' sake.

Lemma 3.2 ((Antoniadis et al., 2023b)). *The total cost incurred by the FtP algorithm is $\sum_{t=1}^n d(s_t, r_t) \leq \sum_{t=1}^n \text{dist}(P_t, P_{t-1} \cup \{r_t\})$.*

3.2. Construction of Virtual Predictions

In this subsection, we present the constructions of virtual predictions and prove Theorem 3.1. Recall that the FtP algorithm requires a prediction in every round while we make

Algorithm 1 Follow-the-Prediction

```

1:  $P_0 \leftarrow \emptyset, S_0 \leftarrow \emptyset$ 
2: for each round  $t \in \{1, \dots, n\}$  do
3:    $P_t \leftarrow$  the prediction in round  $t$ 
4:    $\mu_1 \leftarrow$  a min-cost perfect matching between
      $P_t$  and  $P_{t-1} \cup \{r_t\}$  where every server in
      $P_t \cap (P_{t-1} \cup \{r_t\})$  is matched to itself
5:    $p_t \leftarrow$  the counterpart of  $r_t$  in  $\mu_1$ 
6:    $\mu_2 \leftarrow$  a min-cost perfect matching between
      $S \setminus P_{t-1}$  and  $S \setminus S_{t-1}$  where every server
     in  $S \setminus (P_{t-1} \cup S_{t-1})$  is matched to itself
7:    $s_t \leftarrow$  the counterpart of  $p_t$  in  $\mu_2$ 
8:   Match  $r_t$  with  $s_t$ 
9:    $S_t \leftarrow S_{t-1} \cup \{s_t\}$ 
10: end for
    
```

queries to the oracle only for every k rounds. We therefore need to construct virtual predictions for the remaining rounds.

We achieve this goal by a γ -adherent, strongly ρ -competitive algorithm $\mathcal{A}$ for some $\gamma \geq 1$ and $\rho : \mathbb{N} \rightarrow \mathbb{R}_+$. Intuitively speaking, for the rounds after querying $\hat{P}$ to the oracle until making the next query (or termination), we execute $\mathcal{A}$ as a subroutine given an auxiliary instance where the servers are $S \setminus \hat{P}$ and the requests are those residual after querying $\hat{P}$. We then serve $\hat{P} \cup \hat{S}$ to the FtP algorithm as the prediction in each round, where $\hat{S} \subseteq S \setminus \hat{P}$ denotes the servers matched by $\mathcal{A}$ in the auxiliary instance at the end of each round. A detailed pseudocode is provided in Algorithm 2.

Algorithm 2 Construction of (virtual) predictions

```

1:  $\hat{P} \leftarrow \emptyset$ 
2: Initialize a new instance for  $\mathcal{A}$  with servers  $S$ 
3: for each round  $t \in \{1, \dots, n\}$  do
4:   if  $t$  is a multiple of  $k$  then
5:     Query  $P_t$  to the oracle
6:      $\hat{P} \leftarrow P_t$ 
7:     Initialize a new instance for  $\mathcal{A}$  with servers  $S \setminus \hat{P}$ 
8:   else
9:     Feed  $r_t \in R$  to  $\mathcal{A}$ 
10:    Let  $\hat{S} \subseteq S \setminus \hat{P}$  be the servers matched by  $\mathcal{A}$ 
    until now
11:     $P_t \leftarrow \hat{P} \cup \hat{S}$ 
12:   end if
13: end for
    
```

Our meta-algorithm is then the FtP algorithm (Algorithm 1) with this construction of predictions (Algorithm 2). It is not hard to see that the constructed predictions are feasible, i.e., $P_t \subseteq S$ and $|P_t| = t$ for every $t \in \{1, \dots, n\}$, implying that our algorithm is well-defined.

It remains to show the competitiveness of our algorithm to prove Theorem 3.1. To this end, let us define more notation. Recall that we fix an optimal perfect matching OPT with hindsight. For every $t \in \{1, \dots, n\}$, recall also that $r_t \in R$ is the request arriving in round t , and let o_t denote the server matched with r_t in OPT .

Note that we construct virtual predictions using $\mathcal{A}$ given an auxiliary instance between two consecutive rounds making queries to the oracle. We call the maximal subsequence of rounds for each auxiliary instance a *phase*. More precisely, for $i \in \{0, \dots, \lfloor n/k \rfloor\}$, phase i consists of the rounds $\{ik + 1, \dots, ik + (k - 1)\}$ (or $\{ik + 1, \dots, n\}$ for the last phase). Let $\ell^{(i)}$ denote the length of phase i , i.e., $\ell^{(i)} = n - ik$ if $i = \lfloor n/k \rfloor$ and $\ell^{(i)} = k - 1$ otherwise.

For $i \in \{0, \dots, \lfloor n/k \rfloor\}$, let $\widehat{R}^{(i)} := \{r_{ik+1}, \dots, r_{ik+\ell^{(i)}}\}$ denote the requests fed to $\mathcal{A}$ during phase i , and let $\widehat{\text{OPT}}^{(i)}$ denote a minimum-cost “maximum” matching between $S \setminus P_{ik}$ and $\widehat{R}^{(i)}$ where $\widehat{R}^{(i)}$ is all matched. We first show that the cost of $\widehat{\text{OPT}}^{(i)}$ —an “optimal” matching in the auxiliary instance of phase i —is bounded.

Lemma 3.3. *For each $i \in \{0, \dots, \lfloor n/k \rfloor\}$,*

$$\text{cost}(\widehat{\text{OPT}}^{(i)}) \leq \sum_{j=1}^{\ell^{(i)}} d(o_{ik+j}, r_{ik+j}) + \eta_{ik}.$$

Proof. As the phase is clear from context, we omit the phase in the notation and simply write $\widehat{R}$, ℓ , and $\widehat{\text{OPT}}$. Let $\widehat{P} := P_{ik}$ denote the prediction queried the last to the oracle until then. For every $j \in \{1, \dots, \ell\}$, we further write $\widehat{r}_j := r_{ik+j}$ for notational simplicity. Let $\widehat{o}_j \in S \setminus \widehat{P}$ denote the server matched with $\widehat{r}_j$ in $\widehat{\text{OPT}}$, and let $\widehat{O} \subseteq S \setminus \widehat{P}$ denote the all servers matched with $\widehat{R}$ in $\widehat{\text{OPT}}$; we hence have

$$\text{cost}(\widehat{\text{OPT}}) = \text{dist}(\widehat{O}, \widehat{R}) = \text{dist}(\widehat{O} \cup \widehat{P}, \widehat{R} \cup \widehat{P}),$$

where the last equality is due to Fact 2.2.

We claim that, for any $T \subseteq S$ of size $|T| = |\widehat{O} \cup \widehat{P}| = ik + \ell$, we have

$$\text{dist}(\widehat{O} \cup \widehat{P}, \widehat{R} \cup \widehat{P}) \leq \text{dist}(T, \widehat{R} \cup \widehat{P}). \quad (1)$$

If $T \supseteq \widehat{P}$, the claim immediately follows from the optimality of $\widehat{O}$ and the fact that $\text{dist}(T, \widehat{R} \cup \widehat{P}) = \text{dist}(T \setminus \widehat{P}, \widehat{R})$.

We now consider the case where $T \not\supseteq \widehat{P}$. In this case, we argue that there always exists $T' \supseteq \widehat{P}$ with $|T'| = |T|$ such that $\text{dist}(T', \widehat{R} \cup \widehat{P}) \leq \text{dist}(T, \widehat{R} \cup \widehat{P})$. For any $u \in \widehat{P} \setminus T$, let $v \in T \setminus \widehat{P}$ be the counterpart of u in a minimum-cost perfect matching μ between $T \setminus \widehat{P}$ and $(\widehat{R} \cup \widehat{P}) \setminus T$. Now consider $T' := (T \setminus \{v\}) \cup \{u\}$. Since substituting matching edge $\{u, v\}$ with edge $\{u, u\}$ in μ yields a feasible perfect

matching between T' and $\widehat{R} \cup \widehat{P}$, we have

$$\begin{aligned} \text{dist}(T', \widehat{R} \cup \widehat{P}) &\leq \text{dist}(T, \widehat{R} \cup \widehat{P}) - d(u, v) + d(u, u) \\ &\leq \text{dist}(T, \widehat{R} \cup \widehat{P}) \end{aligned}$$

with $|T' \cap \widehat{P}| > |T \cap \widehat{P}|$. Repeating this substitution completes the proof of this case.

Recall that $O_t \subseteq S$ denotes the servers matched with the first t requests R_t in OPT for every $t \in \{1, \dots, n\}$. We can now derive

$$\begin{aligned} \text{cost}(\widehat{\text{OPT}}) &\leq \text{dist}(O_{ik+\ell}, \widehat{R} \cup \widehat{P}) \\ &\leq \text{dist}(O_{ik+\ell}, O_{ik} \cup \widehat{R}) + \text{dist}(O_{ik} \cup \widehat{R}, \widehat{R} \cup \widehat{P}) \\ &= \sum_{j=1}^{\ell} d(o_{ik+j}, r_{ik+j}) + \eta_{ik}, \end{aligned}$$

where the first inequality comes from Equation (1), and the second from Fact 2.2. $\square$

Notice that, due to Lemma 3.2, the cost of the matching output by the FtP algorithm is bounded by $\sum_{t=1}^n \text{dist}(P_t, P_{t-1} \cup \{r_t\})$. We split this expression into phases and separately bound the contribution of each phase as follows.

Lemma 3.4. *For each $i \in \{0, \dots, \lfloor n/k \rfloor\}$,*

$$\begin{aligned} \mathbb{E} \left[\sum_{j=1}^{\ell^{(i)}} \text{dist}(P_{ik+j}, P_{ik+j-1} \cup \{r_{ik+j}\}) \right] \\ \leq \rho(\ell^{(i)}) \cdot \left(\sum_{j=1}^{\ell^{(i)}} d(o_{ik+j}, r_{ik+j}) + \eta_{ik} \right). \end{aligned}$$

Proof. For each $j \in \{1, \dots, \ell^{(i)}\}$, let $\widehat{s}_j \in S \setminus P_{ik}$ denote the server matched with r_{ik+j} by $\mathcal{A}$ in phase i . Observe that

$$\text{dist}(P_{ik+j}, P_{ik+j-1} \cup \{r_{ik+j}\}) = d(\widehat{s}_j, r_{ik+j}),$$

implying that the left-hand side of the inequality in the lemma statement is precisely the total cost incurred by $\mathcal{A}$ given the auxiliary instance of phase i where $\{r_{ik+1}, \dots, r_{ik+\ell^{(i)}}\}$ are fed as requests. Since $\mathcal{A}$ is strongly ρ -competitive, this cost is bounded from above by $\rho(\ell^{(i)}) \cdot \text{cost}(\widehat{\text{OPT}}^{(i)})$ in expectation. The proof of this lemma then follows from Lemma 3.3. $\square$

Lemma 3.5. *For each $i \in \{1, \dots, \lfloor n/k \rfloor\}$,*

$$\begin{aligned} \text{dist}(P_{ik}, P_{ik-1} \cup \{r_{ik}\}) \\ \leq (1 + \gamma) \sum_{j=1}^k d(o_{(i-1)k+j}, r_{(i-1)k+j}) \\ + (1 + \gamma)\eta_{(i-1)k} + \eta_{ik}. \end{aligned}$$

Proof. Let $\widehat{R} := \widehat{R}^{(i-1)}$ be the requests in phase $(i - 1)$, and let $\widehat{S} \subseteq S \setminus P_{(i-1)k}$ denote the servers matched with $\widehat{R}$

Table 1. γ -adherence and strong ρ -competitiveness of online metric matching algorithms: KP93 (Kalyanasundaram & Pruhs, 1993), KMV94 (Khuller et al., 1994), NR17 (Nayyar & Raghvendra, 2017), RAG18 (Raghvendra, 2018), and BBGN14 (Bansal et al., 2014).

ALGORITHM	METRIC	DET?	γ	$\rho(x)$
KP93, KMV94	GENERAL	YES	1	$2x - 1$
NR17	GENERAL	YES	3	$O(\mu_{\mathbb{M}}(S) \log^2 x)$
RAG18	LINE	YES	3	$O(\log x)$
BBGN14	2-HST	NO	1	$O(\log x)$

by $\mathcal{A}$ in phase $(i - 1)$. We can deduce the following:

$$\begin{aligned}
 & \text{dist}(P_{ik}, P_{ik-1} \cup \{r_{ik}\}) \\
 & \leq \text{dist}(P_{ik}, O_{ik}) + \text{dist}(O_{ik}, O_{(i-1)k} \cup \widehat{R} \cup \{r_{ik}\}) \\
 & \quad + \text{dist}(O_{(i-1)k} \cup \widehat{R} \cup \{r_{ik}\}, O_{(i-1)k} \cup \widehat{S} \cup \{r_{ik}\}) \\
 & \quad + \text{dist}(O_{(i-1)k} \cup \widehat{S} \cup \{r_{ik}\}, P_{ik-1} \cup \{r_{ik}\}) \\
 & = \eta_{ik} + \sum_{j=1}^k d(o_{(i-1)k+j}, r_{(i-1)k+j}) \\
 & \quad + \text{dist}(\widehat{R}, \widehat{S}) + \eta_{(i-1)k} \\
 & \leq \eta_{ik} + \sum_{j=1}^k d(o_{(i-1)k+j}, r_{(i-1)k+j}) \\
 & \quad + \gamma \cdot \text{cost}(\widehat{\text{OPT}}^{(i-1)}) + \eta_{(i-1)k} \\
 & \leq (1 + \gamma) \sum_{j=1}^k d(o_{(i-1)k+j}, r_{(i-1)k+j}) \\
 & \quad + (1 + \gamma) \eta_{(i-1)k} + \eta_{ik},
 \end{aligned}$$

where the first inequality and the equality come from Fact 2.2, the second inequality from γ -adherence of $\mathcal{A}$, and the last inequality from Lemma 3.3. $\square$

We are now ready to prove the main theorem.

Proof of Theorem 3.1. Immediate from Lemmas 3.2, 3.4 and 3.5. $\square$

3.3. Applications

Our meta-algorithm takes as a subroutine a γ -adherent, strongly ρ -competitive algorithm $\mathcal{A}$, and many existing algorithms in the literature satisfy these properties. See Table 1 for a summary on a few algorithms that we use in our learning-augmented algorithm; we prove the adherence and strong competitiveness of these algorithms in Appendix C.

Using the classical deterministic algorithm (Kalyanasundaram & Pruhs, 1993; Khuller et al., 1994) for $\mathcal{A}$ gives the following deterministic algorithm.

Corollary 3.6 (cf. Theorem 1.1). *For a general metric and $k \in \mathbb{N}$, there exists a deterministic learning-augmented algorithm that outputs a perfect matching of cost at most*

$(2k - 1) \text{cost}(\text{OPT}) + 2k \eta(Q)$ with making $\lfloor n/k \rfloor$ queries well-separated with parameter k .

The algorithm presented in Nayyar & Raghvendra (2017) (also in Raghvendra (2018)) gives the following corollaries:

Corollary 3.7. *For a general metric and $k \in \mathbb{N}$, there exists a deterministic learning-augmented algorithm that outputs a perfect matching of cost at most $O(\mu_{\mathbb{M}}(S) \log^2 k) \cdot (\text{cost}(\text{OPT}) + \eta(Q))$ with making $\lfloor n/k \rfloor$ queries well-separated with parameter k , where $\mu_{\mathbb{M}}(S)$ denotes the maximum ratio of the traveling salesman tour and the diameter of any subset of S .*

Corollary 3.8. *For a line metric and $k \in \mathbb{N}$, there exists a deterministic learning-augmented algorithm that outputs a perfect matching of cost at most $O(\log k) \cdot (\text{cost}(\text{OPT}) + \eta(Q))$ with making $\lfloor n/k \rfloor$ queries well-separated with parameter k .*

Due to Bansal et al. (2014), we have the following algorithm for 2-HSTs (see Definition C.4).

Corollary 3.9. *For a metric induced by a 2-HST and $k \in \mathbb{N}$, there exists a randomized learning-augmented algorithm that outputs a perfect matching of expected cost at most $O(\log k) \cdot (\text{cost}(\text{OPT}) + \eta(Q))$ with deterministically making $\lfloor n/k \rfloor$ queries well-separated with parameter k .*

As any metric can be embedded into a 2-HST with $O(\log n)$ expected distortion ((Fakcharoenphol et al., 2004), see Theorem C.5), we further derive the following result.

Corollary 3.10 (cf. Theorem 1.2). *For a general metric and $k \in \mathbb{N}$, there exists a randomized learning-augmented algorithm that outputs a perfect matching of expected cost at most $O(\log n \log k) \cdot (\text{cost}(\text{OPT}) + \eta(Q))$ with deterministically making $\lfloor n/k \rfloor$ queries well-separated with parameter k .*

4. Lower Bounds

We establish lower bounds on competitiveness of parsimonious learning-augmented algorithm for online bipartite matching. Due to space constraints, we defer the whole analysis to Appendix D. We call an oracle that always gives an accurate prediction an *omnipotent oracle*.

For deterministic algorithms, our lower bounds complement that the algorithm guaranteed in Corollary 3.6 is best-possible (up to a small constant factor) when the prediction is accurate in the regimes of well-separated queries to the oracle (and bounded number of predictions).

Theorem 4.1. *No deterministic algorithms making at most B queries to the omnipotent oracle that always gives an accurate prediction can achieve a competitive ratio strictly better than $\frac{2n}{B+1} - 1$ for any metric space.*

Theorem 4.2. *No deterministic algorithm making well-separated queries to the omnipotent oracle with separation parameter k can achieve a competitive ratio strictly better than $2k - 1$ for any metric space.*

For randomized algorithms, we present lower bounds on the class of algorithms making deterministic queries for the two regimes, respectively.

Theorem 4.3. *No randomized algorithm that deterministically makes at most B queries to the omnipotent oracle can achieve a competitive ratio of $1 + o(\frac{\log(n/B)}{B})$ for any metric space.*

Theorem 4.4. *No randomized algorithm making well-separated queries to the omnipotent oracle with separation parameter k can achieve a competitive ratio of $o(\log k)$ for any metric space.*

5. Experiments

This section presents an empirical evaluation of our algorithms. We begin by describing the experimental setup in Section 5.1, followed by a discussion of the experimental results in Section 5.2.

5.1. Experimental Setup

Algorithms We evaluate the following five algorithms in our experiments:

- Ours, the algorithm described in Corollary 3.6;
- Comp, the deterministic $(2n - 1)$ -competitive algorithm due to Kalyanasundaram & Pruhs (1993) and Khuller et al. (1994);
- Comb-Comp, the combination of Ours and Comp as formalized in Theorem 1.1;
- Greedy, the simple greedy algorithm that matches every request to its nearest available server;
- Comb-Greedy, the combination of Ours and Greedy.

We remark that, when setting $k = 1$, Ours is equivalent with the Follow-the-Prediction algorithm of Antoniadis et al. (2023b).

Instance Generation Each instance is defined on a metric space with 200 vertices, consisting of 100 servers and 100 requests. We consider three classes of instances—Line, Plane, and Taxi—which are constructed as follows.

To generate a Line instance, we independently sample 200 numbers uniformly at random from the interval $[0, 1]$, each

representing the location of a vertex on the line. The distance between two vertices is defined as the absolute difference between their corresponding numbers. Among these 200 vertices, we sample 100 uniformly at random without replacement as servers, and sample another 100 uniformly at random with replacement as requests.

An instance of Plane is constructed similarly to Line. The only difference is that each vertex corresponds to a point sampled uniformly at random from the two-dimensional unit square $[0, 1]^2$, and distances are measured using the Euclidean metric, with a modest rounding applied for numerical stability.

Inspired by Feng et al. (2024) and Jin & Ma (2022), Taxi denotes a class of instances constructed from the Chicago Taxi Trips (2013–2023) dataset (City of Chicago, 2024). Specifically, for a randomly sampled time point, the most recent 100 data entries preceding this time are treated as servers, while the earliest 100 entries following it are treated as requests arriving sequentially. Distances are measured using the Manhattan metric. In the interest of space, we defer the full details of Taxi instance construction to Appendix F.

Noisy Prediction Generation Our construction of noisy predictions is parameterized by a noise radius $r \geq 0$. For each round t , let $P_t^* \subseteq S$ denote the accurate prediction at that round. For every server $s \in P_t^*$, we independently select a server q_s uniformly at random from those within distance r of s . Let $Q_t := \{q_s \mid s \in P_t^*\}$ denote the resulting multiset; note that repetitions may occur, i.e., it is possible that $q_{s_1} = q_{s_2}$ for distinct $s_1, s_2 \in P_t^*$. To eliminate such repetitions, we compute a minimum-cost matching between Q_t and S in which every server of Q_t is matched. The prediction $P_t \subseteq S$ for round t is then defined as the set of (distinct) servers in S matched to Q_t . Observe that $\text{dist}(P_t^*, P_t) \leq 2tr$, since the matching $\{(q_s, s) \mid s \in P_t^*\}$ is a feasible matching between Q_t and S and has total cost at most tr .

Evaluation Objectives We consider two types of experiments, each designed to evaluate different aspects of the algorithms. The first type focuses on the average performance of the algorithms as a function of the number of actual predictions used. To eliminate potential bias arising from prediction errors, we use only perfect predictions in this set of experiments. Specifically, for each instance class, we compute the average solution quality—measured as the ratio between the algorithm’s cost and the optimal cost—over 100 independently generated instances, for separation parameters $k \in \{1, \dots, 20\}$.

The second type of experiments aims to evaluate performance under noisy predictions. Since the same level of prediction noise may affect different instances in different

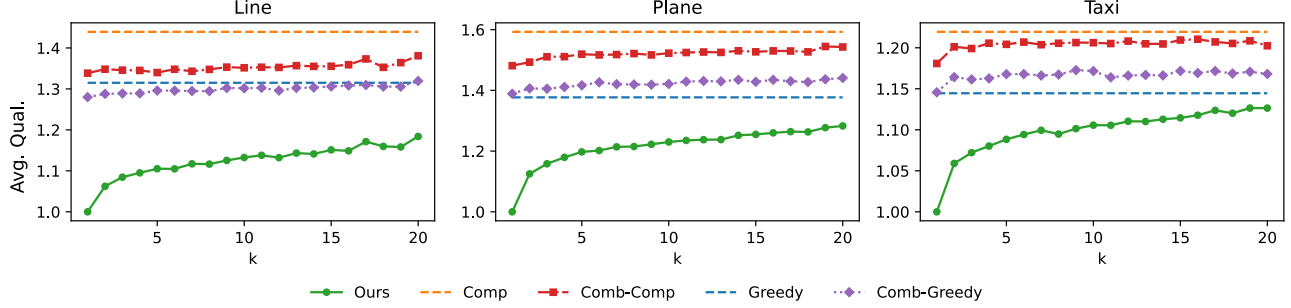


Figure 1. Results of the first type of experiments. The plots for Line, Plane, and Taxi are shown on the left, center, and right, respectively. The x -axis corresponds to the separation parameter k , and the y -axis represents the average solution quality, measured by the ratio of the algorithm’s output cost to the optimal cost with hindsight.

ways, we fix an instance and compute the average solution quality over 100 independently generated predictions. We consider 50 noise radii, uniformly spaced between the minimum non-zero distance and the median non-zero distance, and evaluate performance for separation parameters $k \in \{1, \dots, 20\}$. This set of experiments is respectively conducted on 5 randomly sampled instances for each instance class.

5.2. Experimental Results

The results of the first type of experiments are shown in Figure 1. Overall, the observed trends align well with our expectations. In particular, the performance of Ours, Comb-Comp, and Comb-Greedy degrades as the separation parameter k increases. Among them, the performance of Ours deteriorates more rapidly than that of Comb-Comp and Comb-Greedy, which is expected since combining with Comp or Greedy improves robustness.

On the other hand, Ours consistently outperforms the other algorithms. We attribute this to its ability to fully exploit the available accurate predictions. One notable observation is that Comb-Greedy is outperformed by Greedy on instances of Plane and Taxi. We discuss this behavior at the end of this subsection.

The results of the second type of experiments are summarized in Figure 2. Since these experiments produce 100 plots for each instance class, we present only a representative subset that clearly illustrates the general trends, due to space constraints. The complete set of plots is provided in the Supplementary Material.

Overall, the experimental results again align well with our expectations. For Ours, we observe that the algorithm is more sensitive to prediction noise when the separation parameter k is small. In particular, when $k = 1$, the algorithm achieves near-optimal performance under very low noise, but its performance deteriorates rapidly as the noise level increases. Recall that Ours with $k = 1$ is equivalent to the

FtP algorithm of Antoniadis et al. (2023b). As k increases, the slope of performance degradation becomes noticeably milder, although the performance under accurate predictions also worsens.

This behavior is expected: the more frequently the algorithm exploits predictions, the more strongly it is affected by prediction noise. The combination algorithms exhibit similar qualitative trends, but are substantially more robust to noise due to the presence of the fallback algorithms Comp and Greedy.

We finally discuss the performance of the combination algorithm. Across both types of experiments, the combination algorithm underperforms compared to its fallback algorithm in many cases. This behavior is likely explained by the overhead incurred when switching between the two input algorithms. Consequently, if the performance gap between the input algorithms is relatively narrow, this switching overhead may become significant, potentially leading to worse overall performance for the combination algorithm.

6. Conclusion

In this paper, we extend the study of parsimonious prediction models in learning-augmented online optimization (Im et al., 2022; Sadek & Eliás, 2024) to the setting of online metric matching with action predictions (Antoniadis et al., 2023b).

Several directions remain open for future work. An immediate direction is to develop a more refined analysis for randomized algorithms in the parsimonious setting. In particular, it would be interesting to determine whether one can achieve performance guarantees that depend only on the separation parameter k , rather than the $O(\log n \log k)$ factor in Corollary 3.10, as well as lower bounds for algorithms that make randomized queries, rather than Theorem 4.3.

The combination algorithm is essential for robustifying our algorithmic framework. However, its current version incurs a multiplicative factor of 9 in the competitive ratio. More-

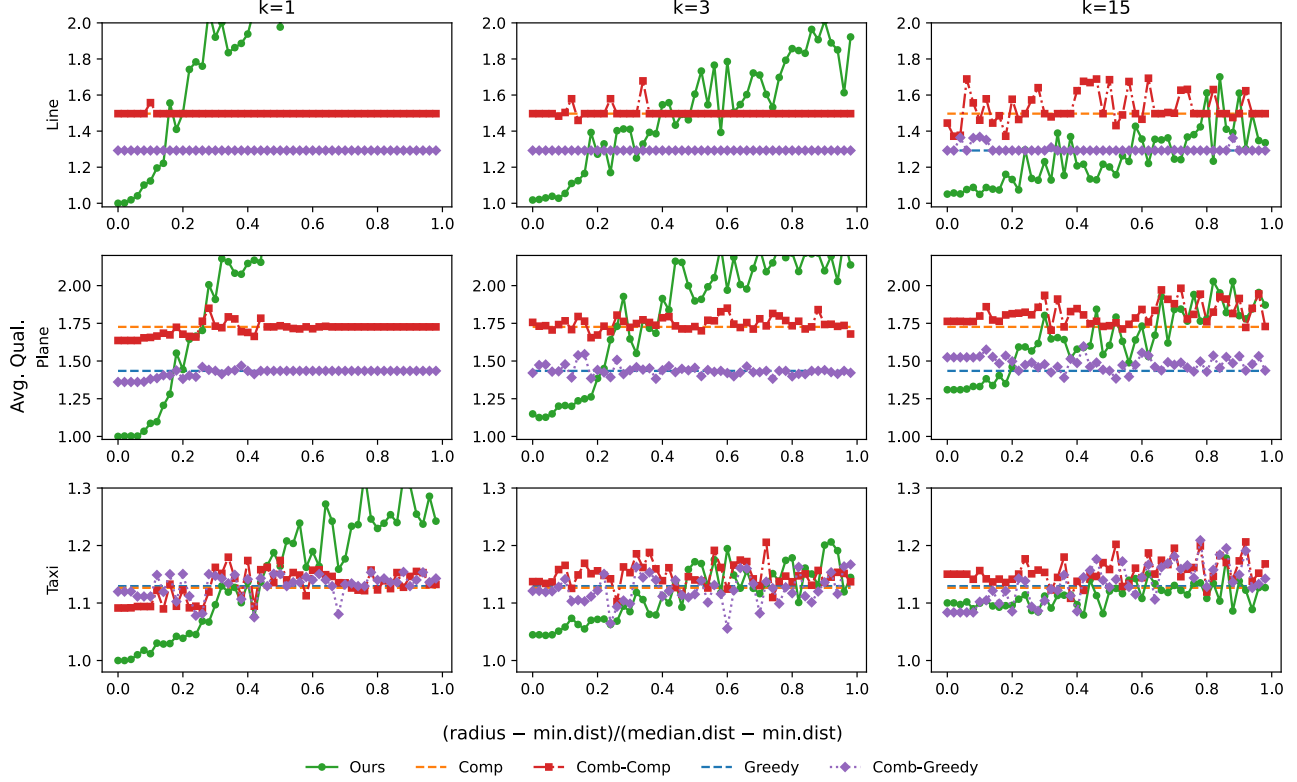


Figure 2. Representative results of the second type of experiments. The rows correspond to the instance classes, while the columns correspond to different values of the separation parameter k . The x -axis displays the normalized prediction noise, represented by $\frac{r - d_{\min}}{d_{\text{med}} - d_{\min}}$, where r is the noise radius, $d_{\min}$ is the minimum non-zero distance, and d_{med} is the median of all non-zero distance values. The y -axis represents the average solution quality, measured by the ratio of the algorithm’s output cost to the optimal cost with hindsight.

over, our experiments show that the combination algorithm often underperforms relative to its fallback algorithm. Designing a theoretically improved and/or practically effective combination algorithm is therefore an important direction for future work.

Another promising direction is to investigate alternative prediction models for online metric matching. One plausible candidate is a variant of the *metric error with outliers* model proposed by Azar et al. (2022), despite the fact that the original model is known not to admit strong guarantees for online metric matching.

Acknowledgements

This work was supported by the NCN grant no. 2020/39/B/ST6/01641. This work was partly supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2021-II212068, Artificial Intelligence Innovation Hub). YS thanks Sungjin Im for introducing this problem and Hyung-Chan An for valuable discussions. The authors are also grateful to Christian Coester

(2026) for acknowledging an inconsistency in Antoniadis et al. (2023b). Part of this work was done while YS was at Yonsei University, South Korea.

Impact Statement

This paper presents theoretical results on parsimonious learning-augmented algorithms for online metric matching, extending prior work on parsimonious learning-augmented online optimization. Given the theoretical nature of this work, we do not anticipate direct negative societal impacts.

References

- Aamand, A., Chen, J., and Indyk, P. (Optimal) online bipartite matching with degree information. *Advances in Neural Information Processing Systems*, 35:5724–5737, 2022.
- Antoniadis, A., Barcelo, N., Nugent, M., Pruhs, K., and Scquizzato, M. A $o(n)$ -competitive deterministic algorithm for online matching on a line. *Algorithmica*, 81(7): 2917–2933, 2019.

- Antoniadis, A., Boyar, J., Eliás, M., Favrholt, L. M., Hoeksma, R., Larsen, K. S., Polak, A., and Simon, B. Paging with succinct predictions. In *International Conference on Machine Learning*, pp. 952–968. PMLR, 2023a.
- Antoniadis, A., Coester, C., Eliáš, M., Polak, A., and Simon, B. Online metric algorithms with untrusted predictions. *ACM transactions on algorithms*, 19(2):1–34, 2023b.
- Antoniadis, A., Gouleakis, T., Kleer, P., and Kolev, P. Secretary and online matching problems with machine learned advice. *Discrete Optimization*, 48:100778, 2023c.
- Azar, Y., Panigrahi, D., and Touitou, N. Online graph algorithms with predictions. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 35–66. SIAM, 2022.
- Bampis, E., Escoffier, B., and Xefteris, M. Parsimonious learning-augmented approximations for dense instances of $\mathcal{NP}$ -hard problems. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 2700–2714. PMLR, 21–27 Jul 2024.
- Bansal, N., Buchbinder, N., Gupta, A., and Naor, J. S. A randomized $O(\log^2 k)$ -competitive algorithm for metric bipartite matching. *Algorithmica*, 68(2):390–403, 2014.
- Benomar, Z. and Perchet, V. Advice querying under budget constraint for online algorithms. *Advances in Neural Information Processing Systems*, 36:75026–75047, 2023.
- Boyar, J., Favrholt, L. M., Kudahl, C., Larsen, K. S., and Mikkelsen, J. W. Online algorithms with advice: A survey. *ACM Computing Surveys (CSUR)*, 50(2):1–34, 2017.
- Choo, D., Gouleakis, T., Ling, C. K., and Bhattacharyya, A. Online bipartite matching with imperfect advice. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 8762–8781, 2024.
- Choo, D., Jin, B., and Shin, Y. Learning-augmented online bipartite fractional matching. *Advances in Neural Information Processing Systems*, 2025.
- City of Chicago. Chicago Taxi Trips. <https://data.cityofchicago.org/Transportation/Taxi-Trips/wrvz-psew>, 2024. Accessed January 16, 2026.
- Coester, C. Personal communication, 2026.
- Drygala, M., Nagarajan, S. G., and Svensson, O. Online algorithms with costly predictions. In *International Conference on Artificial Intelligence and Statistics*, pp. 8078–8101. PMLR, 2023.
- Fakcharoenphol, J., Rao, S., and Talwar, K. A tight bound on approximating arbitrary metrics by tree metrics. *Journal of Computer and System Sciences*, 69:485–497, 2004.
- Feng, Y., Niazadeh, R., and Saberi, A. Two-stage stochastic matching and pricing with applications to ride hailing. *Operations Research*, 72(4):1574–1594, 2024.
- Fiat, A., Rabani, Y., and Ravid, Y. Competitive k-server algorithms. *Journal of Computer and System Sciences*, 48(3):410–428, 1994.
- Fuchs, B., Hochstättler, W., and Kern, W. Online matching on a line. *Theoretical Computer Science*, 332(1-3):251–264, 2005.
- Gupta, A. and Lewi, K. The online metric matching problem for doubling metrics. In *International Colloquium on Automata, Languages, and Programming*, pp. 424–435. Springer, 2012.
- Gupta, A., Guruganesh, G., Peng, B., and Wajc, D. Stochastic online metric matching. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, pp. 67–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2019.
- Im, S., Kumar, R., Petety, A., and Purohit, M. Parsimonious learning-augmented caching. In *International Conference on Machine Learning*, pp. 9588–9601. PMLR, 2022.
- Jin, B. and Ma, W. Online bipartite matching with advice: Tight robustness-consistency tradeoffs for the two-stage model. *Advances in Neural Information Processing Systems*, 35:14555–14567, 2022.
- Kalyanasundaram, B. and Pruhs, K. Online weighted matching. *Journal of Algorithms*, 14(3):478–488, 1993.
- Khuller, S., Mitchell, S. G., and Vazirani, V. V. On-line algorithms for weighted bipartite matching and stable marriages. *Theoretical Computer Science*, 127(2):255–267, 1994.
- Kumar, R., Purohit, M., and Svitkina, Z. Improving online algorithms via ml predictions. *Advances in Neural Information Processing Systems*, 31:9684–9693, 2018.
- Lindermayr, A. and Megow, N. Algorithms with Predictions. <https://algorithms-with-predictions.github.io>.
- Lykouris, T. and Vassilvitskii, S. Competitive caching with machine learned advice. *Journal of the ACM (JACM)*, 68(4):1–25, 2021.
- Mahdian, M., Nazerzadeh, H., and Saberi, A. Online optimization with uncertain information. *ACM Transactions on Algorithms (TALG)*, 8(1):1–29, 2012.

- Meyerson, A., Nanavati, A., and Poplawski, L. Randomized online algorithms for minimum metric bipartite matching. In *Proceedings of the seventeenth annual ACM-SIAM symposium on Discrete algorithm*, pp. 954–959, 2006.
- Mitzenmacher, M. and Vassilvitskii, S. Algorithms with predictions. *Communications of the ACM*, 65(7):33–35, 2022.
- Nayyar, K. and Raghvendra, S. An input sensitive online algorithm for the metric bipartite matching problem. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 505–515. IEEE, 2017.
- Peserico, E. and Scquizzato, M. Matching on the line admits no $o(\sqrt{\log n})$ -competitive algorithm. *ACM Transactions on Algorithms*, 19(3):28:1–4, 2023.
- Raghvendra, S. A robust and optimal online algorithm for minimum metric bipartite matching. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2016)*, pp. 18–1. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2016.
- Raghvendra, S. Optimal analysis of an online algorithm for the bipartite matching problem on a line. In *34th International Symposium on Computational Geometry (SoCG 2018)*, pp. 67–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018.
- Sadek, K. A. and Eliás, M. Algorithms for caching and MTS with reduced number of predictions. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- Spaeh, F. and Ene, A. Online ad allocation with predictions. *Advances in Neural Information Processing Systems*, 36: 17265–17295, 2023.

A. Further Related Work

Online Metric Matching Online metric matching has been studied under a variety of metric spaces. Gupta & Lewi (2012) presented a randomized $O(\log n)$ -competitive algorithm for line and doubling metrics. For deterministic algorithms, Antoniadis et al. (2019) gave the first sublinear-competitive algorithm for the line metric. Subsequently, Raghvendra (2018) developed a deterministic $O(\log n)$ -competitive algorithm for line metrics, building on an instance-dependent $O(\mu_{\mathbb{M}}(S) \log^2 n)$ -competitive algorithm due to Nayyar & Raghvendra (2017), where $\mu_{\mathbb{M}}(S)$ denotes the maximum ratio between the traveling salesman tour and the diameter over all subsets of the server set S . On the lower-bound side, Peserico & Scquizzato (2023) recently showed that any algorithm, including randomized ones, must have a competitive ratio of at least $\Omega(\sqrt{\log n})$ on the line metric, improving upon the previous lower bound of 9.001 for deterministic algorithms established by Fuchs et al. (2005).

The problem has also been studied under other arrival models than the adversarial model considered in this paper. Raghvendra (2016) considered the random arrival model and obtained a best-possible $O(\log n)$ -competitive algorithm, together with a matching lower bound. In the known i.i.d. model—where each request is drawn independently from a known distribution—Gupta et al. (2019) presented an $O((\log \log \log n)^2)$ -competitive algorithm for general metrics, which achieves a constant competitive ratio of 9 for line and tree metrics.

Parsimonious Usage of Predictions Besides Im et al. (2022) and Sadek & Eliás (2024), a growing body of work has investigated how to reduce the amount of predictive information used in learning-augmented algorithms. Drygala et al. (2023) studied the ski rental and Bahncard problems in settings where using predictions incurs an additional cost. Benomar & Perchet (2023) examined the tradeoff between algorithmic performance and a bounded number of predictions for problems such as ski rental, secretary, and non-clairvoyant scheduling, under a model in which predictions are accurate with known probabilities. Bampis et al. (2024) showed that a logarithmic number of predictions suffices to improve the running time of PTASs for dense instances of several NP-hard problems. Antoniadis et al. (2023a) introduced single-bit prediction models for paging and developed learning-augmented algorithms for both models, together with matching lower bounds. Finally, we note that the parsimonious use of predictions in learning-augmented algorithms is closely related to the literature on *online optimization with advice*, which studies the bit complexity of accurate advice required to achieve given performance guarantees; we refer interested readers to the survey by Boyar et al. (2017).

Learning-Augmented Online Matching and Beyond Online bipartite matching has also been extensively studied in the learning-augmented setting. Mahdian et al. (2012) initiated this line of work by developing a learning-augmented algorithm for the AdWords problem under the small-bids assumption; their result was later improved by Choo et al. (2025). Spaeh & Ene (2023) extended this framework to the Display Ads and generalized assignment problems. Jin & Ma (2022) investigated a two-stage model of unweighted/vertex-weighted/edge-weighted online bipartite matching. Related work has also considered the random arrival model (Antoniadis et al., 2023c; Choo et al., 2024) and the stochastic graph setting (Aamand et al., 2022). Beyond online bipartite matching, a substantial body of research has explored learning-augmented algorithms more broadly; interested readers are referred to the survey by Mitzenmacher & Vassilvitskii (2022) and the website (Lindermayr & Megow), which provides a comprehensive collection of publications in the area.

B. Combination Algorithm

In this appendix, we prove Theorem 2.3.

Algorithm Description We first introduce some notation: For an online algorithm $\mathcal{C}$ and any $t \in \{1, \dots, n\}$, let $M_t^{\mathcal{C}}$ be the intermediate matching that $\mathcal{C}$ maintains at the end of round t . Let $s_t^{\mathcal{C}} \in S$ and $S_t^{\mathcal{C}} \subseteq S$ denote the server matched by $\mathcal{C}$ at round t and the set of servers matched by $\mathcal{C}$ so far, respectively. Let $S_0^{\mathcal{C}} = \emptyset$.

We now describe the combination algorithm. For every round $t \in \{1, \dots, n\}$, let $S_t \subseteq S$ denote the servers matched by this combination algorithm at the end of round t ; let $S_0 = \emptyset$. Recall that we are given two online algorithms $\mathcal{A}$ and $\mathcal{B}$. We divide the entire rounds into *phases*, where we try to follow $\mathcal{A}$ in every odd phase, and $\mathcal{B}$ in every even phase.

At the beginning of a round $t \in \{1, \dots, n\}$, let us assume that we are in phase i for some odd i . We first determine if the cost incurred by $\mathcal{A}$ is greater than 2^i or not. If it is the case (i.e., $\text{cost}(M_t^{\mathcal{A}}) > 2^i$), we move to the next phase without processing the request of this round. Here we remark that no rounds can be processed in a phase.

Otherwise, if $\text{cost}(M_{t-1}^A) \leq 2^i$, let s_t denote the counterpart of s_t^A in a minimum-cost perfect matching μ_t between $S \setminus S_{t-1}^A$ and $S \setminus S_{t-1}$, where every $s \in S \setminus (S_{t-1} \cup S_{t-1}^A)$ is matched with itself. We then match r_t with s_t . For an even phase, we replace every $\mathcal{A}$ with $\mathcal{B}$ in the description. This is the end of the description of the combination algorithm.

The pseudocode of the combination algorithm is provided in Algorithm 3.

Algorithm 3 Combination Algorithm

```

 $i \leftarrow 1, t \leftarrow 1, S_0 \leftarrow \emptyset$ 
while  $t \leq n$  do
     $\mathcal{C} \leftarrow \begin{cases} \mathcal{A}, & \text{if } i \text{ is odd} \\ \mathcal{B}, & \text{if } i \text{ is even} \end{cases}$ 
    if  $\text{cost}(M_t^{\mathcal{C}}) > 2^i$  then
         $i \leftarrow i + 1$ 
    else
         $s_t^{\mathcal{C}} \leftarrow$  the server matched by  $\mathcal{C}$  at round  $t$ 
         $\mu_t \leftarrow$  a minimum-cost perfect matching between  $S \setminus S_{t-1}^{\mathcal{C}}$  and  $S \setminus S_{t-1}$ , where every server in  $S \setminus (S_{t-1}^{\mathcal{C}} \cup S_{t-1})$ 
            is matched with itself
         $s_t \leftarrow$  the counterpart of  $s_t^{\mathcal{C}}$  in  $\mu_t$ 
        Match  $r_t$  with  $s_t$ 
         $S_t \leftarrow S_{t-1} \cup \{s_t\}$ 
         $t \leftarrow t + 1$ 
    end if
end while
    
```

Analysis We now analyze the combination algorithm.

Lemma B.1. *The combination algorithm outputs a perfect matching.*

Proof. Fix any round t . Recall that any server in $S \setminus S_{t-1}^{\mathcal{C}}$ is matched with a server in $S \setminus S_{t-1}$ in μ_t . Note that $s_t^{\mathcal{C}} \in S \setminus S_{t-1}^{\mathcal{C}}$, implying that $s_t \in S \setminus S_{t-1}$. $\square$

Lemma B.2. *For any $t \in \{1, \dots, n\}$, $d(s_t, s_t^{\mathcal{C}}) = \text{dist}(S_{t-1}, S_{t-1}^{\mathcal{C}}) - \text{dist}(S_t, S_t^{\mathcal{C}})$.*

Proof. As shown in the proof of Lemma B.1, we have $s_t \in S \setminus S_{t-1}$ and $s_t^{\mathcal{C}} \in S \setminus S_{t-1}^{\mathcal{C}}$. Moreover, since μ_t is a minimum-cost perfect matching between $S \setminus S_{t-1}$ and $S \setminus S_{t-1}^{\mathcal{C}}$ in which s_t and $s_t^{\mathcal{C}}$ are matched together, we have

$$\begin{aligned}
 d(s_t, s_t^{\mathcal{C}}) &= \text{dist}(S \setminus S_{t-1}, S \setminus S_{t-1}^{\mathcal{C}}) - \text{dist}(S \setminus (S_{t-1} \cup \{s_t\}), S \setminus (S_{t-1}^{\mathcal{C}} \cup \{s_t^{\mathcal{C}}\})) \\
 &= \text{dist}(S_{t-1}, S_{t-1}^{\mathcal{C}}) - \text{dist}(S_t, S_t^{\mathcal{C}}).
 \end{aligned}$$

$\square$

Lemma B.3. *If $\text{cost}(\text{OPT}) \geq 1$, the cost incurred by the combination algorithm is at most $9 \min\{\text{cost}(\mathcal{A}), \text{cost}(\mathcal{B})\}$.*

Proof. If there is only a single phase throughout the execution, the cost incurred by the algorithm is exactly

$$\text{cost}(\mathcal{A}) \leq 2 \leq 2 \min\{\text{cost}(\mathcal{A}), \text{cost}(\mathcal{B})\},$$

where the second inequality is due to the assumption that $\text{cost}(\text{OPT}) \geq 1$.

We now assume that the number of phases is at least 2. Let $\ell \geq 2$ denote the last (nonempty) phase. For each phase i , let τ_i be the last round of the phase; if a phase i is empty, then we have $\tau_i = \tau_{i-1}$. Let $\tau_0 := 0$ for consistency. By the definition of the combination algorithm, we have

$$\text{cost}(M_{\tau_i}^{\mathcal{A}}) \leq 2^i \text{ for odd } i \leq \ell \text{ and } \text{cost}(M_{\tau_i}^{\mathcal{B}}) \leq 2^i \text{ for even } i \leq \ell. \quad (2)$$

Moreover, for an odd i , the total cost incurred during phase i can be bounded as follows:

$$\begin{aligned}
 \sum_{t=\tau_{i-1}+1}^{\tau_i} d(s_t, r_t) &\leq \sum_{t=\tau_{i-1}+1}^{\tau_i} \left[d(s_t, s_t^A) + d(s_t^A, r_t) \right] \\
 &\leq \sum_{t=\tau_{i-1}+1}^{\tau_i} \left[d(s_t^A, r_t) + \text{dist}(S_{t-1}, S_{t-1}^A) - \text{dist}(S_t, S_t^A) \right] \\
 &= \sum_{t=\tau_{i-1}+1}^{\tau_i} d(s_t^A, r_t) + \text{dist}(S_{\tau_{i-1}}, S_{\tau_{i-1}}^A) - \text{dist}(S_{\tau_i}, S_{\tau_i}^A) \\
 &\leq \sum_{t=\tau_{i-1}+1}^{\tau_i} d(s_t^A, r_t) + \text{dist}(S_{\tau_{i-1}}^A, R_{\tau_{i-1}}) + \text{dist}(S_{\tau_{i-1}}^B, R_{\tau_{i-1}}) + \text{dist}(S_{\tau_{i-1}}, S_{\tau_{i-1}}^B) - \text{dist}(S_{\tau_i}, S_{\tau_i}^A) \\
 &\leq \sum_{t=\tau_{i-1}+1}^{\tau_i} d(s_t^A, r_t) + \text{cost}(M_{\tau_{i-1}}^A) + \text{cost}(M_{\tau_{i-1}}^B) + \text{dist}(S_{\tau_{i-1}}, S_{\tau_{i-1}}^B) - \text{dist}(S_{\tau_i}, S_{\tau_i}^A) \\
 &= \text{cost}(M_{\tau_i}^A) + \text{cost}(M_{\tau_{i-1}}^B) + \text{dist}(S_{\tau_{i-1}}, S_{\tau_{i-1}}^B) - \text{dist}(S_{\tau_i}, S_{\tau_i}^A), \tag{3}
 \end{aligned}$$

where the second inequality is due to Lemma B.2. For an even i , the symmetric derivation gives

$$\sum_{t=\tau_{i-1}+1}^{\tau_i} d(s_t, r_t) \leq \text{cost}(M_{\tau_i}^B) + \text{cost}(M_{\tau_{i-1}}^A) + \text{dist}(S_{\tau_{i-1}}, S_{\tau_{i-1}}^A) - \text{dist}(S_{\tau_i}, S_{\tau_i}^B). \tag{4}$$

For presentational simplicity, let us assume that ℓ is odd; a symmetric argument proves the other case. Note that, by Equations (3) and (4), the total cost incurred by the combination algorithm is bounded by

$$\begin{aligned}
 &2 \cdot \left[\sum_{\text{odd } i < \ell} \text{cost}(M_{\tau_i}^A) + \sum_{\text{even } i < \ell} \text{cost}(M_{\tau_i}^B) \right] + \text{cost}(M_{\tau_\ell}^A) + \text{dist}(S_0, S_0^B) - \text{dist}(S_n, S_n^A) \\
 &\leq 2 \sum_{i=1}^{\ell-1} 2^i + \text{cost}(\mathcal{A}) \leq \text{cost}(\mathcal{A}) + 2^{\ell+1}, \tag{5}
 \end{aligned}$$

where the first inequality follows from Equation (2) and the fact that $\text{dist}(S_0, S_0^B) = \text{dist}(S_n, S_n^A) = 0$.

We now determine the factor. If $\text{cost}(\mathcal{A}) \leq \text{cost}(\mathcal{B})$, we can further bound Equation (5) by

$$\left(1 + \frac{2^{\ell+1}}{\text{cost}(\mathcal{A})}\right) \text{cost}(\mathcal{A}) < \left(1 + \frac{2^{\ell+1}}{2^{\ell-2}}\right) \min\{\text{cost}(\mathcal{A}), \text{cost}(\mathcal{B})\} = 9 \cdot \min\{\text{cost}(\mathcal{A}), \text{cost}(\mathcal{B})\},$$

where the inequality is due to the fact that phase $\ell - 2$ terminates when $\text{cost}(M_{\tau_{\ell-2}+1}^A) > 2^{\ell-2}$. On the other hand, if $\text{cost}(\mathcal{A}) > \text{cost}(\mathcal{B})$, Equation (5) can be bounded from above by

$$\left(\frac{\text{cost}(\mathcal{A}) + 2^{\ell+1}}{\text{cost}(\mathcal{B})}\right) \text{cost}(\mathcal{B}) < \left(\frac{2^\ell + 2^{\ell+1}}{2^{\ell-1}}\right) \min\{\text{cost}(\mathcal{A}), \text{cost}(\mathcal{B})\} = 6 \cdot \min\{\text{cost}(\mathcal{A}), \text{cost}(\mathcal{B})\},$$

where the inequality follows from Equation (2) and the fact that phase $\ell - 1$ terminates when $\text{cost}(M_{\tau_{\ell-1}+1}^B) > 2^{\ell-1}$. $\square$

Following is a simple fact that will be used in the proof of the main theorem.

Fact B.4. For two random variables X and Y , we have $\mathbb{E}[\min\{X, Y\}] \leq \min\{\mathbb{E}[X], \mathbb{E}[Y]\}$.

Proof. For any realization of Y , we have $\min\{X, Y\} \leq X$, implying that $\mathbb{E}[\min\{X, Y\}] \leq \mathbb{E}[X]$. A symmetric argument shows $\mathbb{E}[\min\{X, Y\}] \leq \mathbb{E}[Y]$. $\square$

We are now ready to prove the main theorem.

Proof of Theorem 2.3. Due to Lemma B.1, the combination algorithm is guaranteed to output a perfect matching. From Lemma B.3, we can infer that the expected cost incurred by the combination algorithm is at most $9 \cdot \mathbb{E}[\min\{\text{cost}(\mathcal{A}), \text{cost}(\mathcal{B})\}]$. Fact B.4 then completes the proof. $\square$

C. Adherence and Strong Competitiveness of Existing Algorithms

We prove the adherence and strong competitiveness of the algorithms in Table 1.

C.1. Deterministic Algorithms

All the deterministic algorithms presented in Table 1 can be described in the framework of Nayyar & Raghvendra (2017).

We first need to define the γ -net-cost of an augmenting path.

Definition C.1 ((Nayyar & Raghvendra, 2017)). For any matching M and a parameter $\gamma \geq 1$, the γ -net-cost of any augmenting path P is defined as

$$\gamma \cdot \left(\sum_{e \in P \setminus M} d(e) \right) - \sum_{e \in P \cap M} d(e).$$

We can now describe the framework: It maintains two matchings M^{on} and M^{off} satisfying:

- no reassignments happen in M^{on} , whereas it happens in M^{off} ;
- the matched servers are equivalent between M^{on} and M^{off} .

Upon arrival of a request $r_t \in R$, the framework computes an augmenting path P_t from r_t to any unmatched server s_t that achieves the minimum γ -net cost. It then augments M^{off} along P_t , and matches r_t with s_t in M^{on} . This is the end of the framework description.

Note that the deterministic $(2n - 1)$ -competitive algorithm is equivalent with the framework with $\gamma = 1$ (Kalyanasundaram & Pruhs, 1993; Khuller et al., 1994). Nayyar & Raghvendra (2017) and Raghvendra (2018) define $\gamma = 3$.

The proof for adherence of this framework can be found in Nayyar & Raghvendra (2017).

Lemma C.2. *This framework is γ -adherent.*

Proof. It is shown in (Nayyar & Raghvendra, 2017) that the framework always maintains M^{off} and a dual solution $y \in \mathbb{R}_+^{S \cup R}$ to be γ -feasible; that is, in any round,

- $y(s) + y(r) \leq \gamma \cdot d(s, r)$ for all $(s, r) \in S \times R$;
- $y(s) + y(r) = d(s, r)$ for all $(s, r) \in M^{\text{off}}$;
- $y(s) = 0$ and $y(r) = 0$ for all unmatched $s \in S$ and not-yet-arrived $r \in R$.

Note that $\text{cost}(M^{\text{off}}) = \sum_{v \in S \cup R} y(v)$ while y/γ is feasible to the dual of the weighted bipartite matching LP relaxation. This completes the lemma. $\square$

The strong competitiveness of this algorithm can be easily derived from the adherence and determinism.

Lemma C.3. *For any $\rho : \mathbb{N} \rightarrow \mathbb{R}$, if this framework is ρ -competitive, it is strongly $\gamma\rho$ -competitive.*

Proof. For any round t , let S_t denote the servers matched by this framework up to and including round t . Consider an auxiliary instance (S, d, S_t, R_t) . The framework is guaranteed to output a perfect matching between S_t and R_t of cost at most $\rho(t) \cdot \text{dist}(S_t, R_t)$. Observe that the execution of this framework given (V, d, S, R) up to round t is equivalent with that given (V, d, S_t, R_t) until termination. The lemma then immediately follows from Lemma C.2. $\square$

The above two lemmas together complete the proof of adherence and strong competitiveness of the deterministic algorithms in Table 1.

C.2. Randomized Algorithm

As shown in [Meyerson et al. \(2006\)](#) and [Bansal et al. \(2014\)](#), it is without loss of generality to assume that the vertices of the given metric spaces are precisely the servers (i.e., $V = S$) because one can easily modify any general metric space into such one at a constant factor in competitive ratios. [Bansal et al. \(2014\)](#) presented an algorithm on 2-HSTs.

Definition C.4 (HST). Given $\alpha \geq 1$, an α -hierarchical well-separated tree (α -HST) is a rooted tree $T = (U, E)$ along with a length $\ell : E \rightarrow \mathbb{R}_+$ satisfying

- for each internal node $u \in U$, the length between u and any child of u is identical, i.e., $\ell(u, c_1) = \ell(u, c_2)$ for any two children c_1 and c_2 of u ;
- for any internal node $u \in U$, the length between u and its parent p is exactly α times the length between u and its child c , i.e., $\ell(u, p) = \alpha \cdot \ell(u, c)$.

It is well-known that any metric space can be embedded into an HST with a logarithmic expected distortion:

Theorem C.5 ([\(Fakcharoenphol et al., 2004\)](#)). *Given any metric (V, d) with n vertices, there exists a randomized algorithm producing 2-HST (T, ℓ) such that*

- the leaf nodes of T are corresponding to V ;
- for any $u, v \in V$, the distance between u and v in (T, ℓ) is at least $d(u, v)$;
- for any $u, v \in V$, the expected distance between u and v in (T, ℓ) is bounded from above by $O(\log n) \cdot d(u, v)$.

The factor of $d(u, v)$ in the last property is called the (expected) distortion of (T, ℓ) .

We now describe the algorithm of [Bansal et al. \(2014\)](#). Given a 2-HST $(T = (U, E), \ell)$ with n leaves, for every leaf node $u \in U$ and height $h \geq 0$, let $T(u, h)$ denote the set of leaf nodes that belong to the subtree of height h containing u , and let $N(u, h) := T(u, h) \setminus T(u, h-1)$ (and $N(u, 0) := T(u, 0)$ for $h = 0$). The algorithm maintains two matchings M^{on} and M^{off} such that

- no reassignment happen in M^{on} , but it happens in M^{off} ;
- the matched servers are equivalent between M^{on} and M^{off} .

Throughout the execution, the algorithm maintains the *level* of every server $s \in S$, denoted by $L(s)$, defined as the minimum height of a subtree containing both s and its matched request in M^{off} (and $L(s) := \infty$ if unmatched). Accordingly, it also maintains the level of every request r that has arrived so far, defined as the level of its matched server in M^{off} .

Upon arrival of a request $r_t \in R$, the algorithm modifies M^{off} by the following recursive procedure: Let r be the request to be (re)assigned, initially $r := r_t$ and $L(r) := 0$. It finds the lowest height $h \geq L(r)$ where there is a server $s \in N(r, h)$ whose current level is higher than h , i.e., $L(s) > h$. It then chooses a server s' uniformly at random among the servers s in $N(r, h)$ with $L(s) > h$, and (re)assign s' to r in M^{off} with setting $L(s') = L(r) = h$. If s' was matched with another request r' in M^{off} , the algorithm recursively reassigns r' using the procedure. Otherwise, if s' was unmatched, it then matches r_t with s' in M^{on} and proceeds to the next round.

Its exact adherence is immediate from the proofs of Lemma 4.1 and 4.3 in [Bansal et al. \(2014\)](#), so we omit its proof here.

Lemma C.6. *This algorithm is 1-adherent.*

The strong competitiveness can be shown by a modest adaptation of the proof of Lemma 4.4 in [Bansal et al. \(2014\)](#).

Lemma C.7. *For any round t , the expected cost of the matching maintained by the algorithm at round t is at most $O(\log t) \cdot \min_{M \in \mathcal{M}_t} \text{cost}(M)$.*

Proof. We follow the same line of the proof of Lemma 4.4 in [Bansal et al. \(2014\)](#). Consider any request $r \in R_t$, and let $L(r)$ denote the last level of r at termination. It suffices to show that, for every height $h \in \{0, \dots, L(r)\}$, the expected

number of times r is (re)assigned to servers in $N(r, h)$ is bounded by $O(\log t)$. Assume a round during the execution up to and including round t where r is (re)assigned to a server in $N(r, h)$, and let $W \subseteq N(r, h)$ be the servers in $N(r, h)$ with $L(s) > h$. Observe that W never increases throughout the execution. Moreover, since r can be reassigned only when the newly arrived request r' is closer to the server currently matched to r , all servers in W are also potential servers to be reassigned upon arrival of r' , implying that the probability of the reassignment of r is at most $1/|W|$ for the current W . Finally, if r is reassigned, W decreases by 1. Since we only consider the execution up to round t , we can therefore conclude that the expected number of times r is reassigned to $N(r, h)$ is at most

$$\sum_{i=1}^{\min\{t, |W|\}} \frac{1}{|W| - i + 1} \leq H_t,$$

where W here denotes the snapshot of W when r is (re)assigned to a server in $N(r, h)$ for the first time and H_t is the t -th Harmonic number. The rest of the argument is identical to the proof of Lemma 4.4 in [Bansal et al. \(2014\)](#). $\square$

D. Deferred Lower Bound Analysis

This appendix is devoted to proving the lower bound results in Section 4. In particular, we present adversaries that bound from below the (expected) cost any deterministic (or randomized) algorithm must incur even if the oracle always provide accurate predictions. For the sake of analysis, we define problem setting $\mathcal{T}_n$ as a setting that has a metric space induced from a star graph with n leaf nodes and one center node, and a set of n servers located at n leaf nodes.

D.1. Deterministic Algorithms

Proof of Theorem 4.1. Consider a problem setting $\mathcal{T}_n$. As $\mathcal{A}$ is deterministic, we can assume an adaptive adversary. Before describing the adversary, we first introduce a couple of definitions. In what follows, the adversary will reveal at most one request at each leaf; we say a server is *requested* if a request is revealed at the same location as the server. Moreover, upon a prediction query from $\mathcal{A}$, the adversary will construct a prediction from the servers matched by $\mathcal{A}$ so far plus one server unmatched by $\mathcal{A}$ then; we say a server is *predicted* if it has been an unmatched server of the prediction queried in any round.

We can now describe the adversary: In every round $t \in \{1, \dots, n\}$, if there exists a server $s \in S$ that is matched by $\mathcal{A}$, unpredicted, and unrequested, the adversary reveals a request at the location of s ; otherwise, if there does not exist such a server, the adversary reveals a request at the center. For the construction of prediction P_t in round t (in case where $\mathcal{A}$ makes a query), if there exists a predicted, unmatched server p , the adversary sets $P_t := S_{t-1} \cup \{p\}$, where S_{t-1} is a set of the servers matched by $\mathcal{A}$ at the beginning of round t ; otherwise, if such a server p does not exist, the adversary selects any unmatched server as p and defines $P_t := S_{t-1} \cup \{p\}$.

Observe that the adversary guarantees that at most one predicted, unmatched server p exists throughout the execution while the adversary reveals a new request at the center only when p got matched by $\mathcal{A}$ in the previous round. This observation implies that the number of the requests located at the center is at most $B + 1$, and hence, $\text{cost}(\text{OPT}) \leq B + 1$. On the other hand, since $\mathcal{A}$ incurs two units to serve every request located at a leaf, we can derive

$$\text{cost}(\mathcal{A}) \geq 2n - B - 1 \geq \left(\frac{2n}{B + 1} - 1 \right) \text{cost}(\text{OPT}).$$

$\square$

Proof of Theorem 4.2. Consider a problem setting $\mathcal{T}_n$. The adversary first sends a request to the center in the first round. In each subsequent round, the adversary issues a request to the location of the server most recently matched by the online algorithm. This process continues until the k -th round, at which point the algorithm receives a prediction revealing which server is matched to the center in the offline solution. From that point on, the adversary can no longer continue this strategy.

In the remaining rounds, the adversary issues requests to arbitrary leaf nodes that have not yet received any request.

This construction ensures that the online algorithm incurs a cost of at least $2k - 1$, while the offline algorithm incurs a cost of exactly 1. Consequently, the competitive ratio is at least $2k - 1$. $\square$

D.2. Randomized Algorithms

Lemma D.1 (see, e.g., (Meyerson et al., 2006)). *For the problem setting $\mathcal{T}_n$, any algorithm can incur cost of at least $2H_n - 1$ in expectation, while the minimum cost of a perfect matching with hindsight is 1.*

Proof. The strategy for the adversary is as follows:

1. In the first round, send a request to the center node.
2. For the remaining rounds, uniformly at random pick a node that has not yet received any request and then send a request to that node.

For each round $t \in \{2, \dots, n\}$, the probability that the server co-located with r_t that has already been matched by the algorithm is $\frac{1}{n-t+2}$. Hence, the total expected cost is $1 + 2 \sum_{t=2}^n \frac{1}{n-t+2} = 2H_n - 1$. It is easy to see that the minimum cost of a perfect matching with hindsight is always 1. $\square$

We are now ready to prove Theorem 4.3, our lower bound under the regime of bounded number of predictions.

Proof of Theorem 4.3. Consider the problem setting $\mathcal{T}_n$. The adversary follows the strategy in Lemma D.1 until the algorithm makes a query to the prediction oracle. Because the prediction is perfectly accurate, the algorithm then learns which server is matched to the first request at the center and can subsequently match the most recent request to that server.

To ensure that the algorithm continues to incur nonzero cost, the adversary restarts the strategy in Lemma D.1 from the beginning whenever the algorithm makes a query to the prediction oracle, and repeats this process until the next query is made. This strategy is possible since the algorithm deterministically makes the queries to the oracle. Note that any server that has appeared in a prediction is treated as having received a request, since it is either destined to be matched to a request at the center node or already gets a request.

Let q be the number of queries the algorithm makes to the oracle excluding one made in the n -th round. Then, $q + 1$ requests are sent to the center. Hence, the offline algorithm pays cost of $q + 1$. However, the online algorithm pays a cost of one for the request at the center and expected cost of $\frac{2}{j}$ for an event that a request is sent to an already occupied server in round $n - j + 1$. Thus, the algorithm incurs a cost of at least $q + 1 + \sum_{j=q+2}^n \frac{2}{j} = q + 1 + 2(H_n - H_{q+1})$. Since $q \leq B$, the competitive ratio is at least $1 + \frac{2(H_k - H_{q+1})}{q+1} \geq 1 + \frac{2(H_k - H_{B+1})}{B+1}$. $\square$

We now prove Theorem 4.4.

Proof of Theorem 4.4. Consider the problem setting $\mathcal{T}_n$. Suppose that n is a multiple of k . For the first $n - k$ rounds, the adversary sends the requests to only leaf nodes. Since n is a multiple of k , the prediction is given to the algorithm in the $(n - k)$ -th round. After that, in the $(n - k + 1)$ -th round, the adversary sends a request to the center node and follows the strategy in Lemma D.1. Because both the offline and online algorithms incur zero cost during the first $n - k$ rounds, and because the remaining free servers together with the center node form a setting $\mathcal{T}_k$ for the last k rounds, this yields an expected lower bound of $2H_k - 1$. $\square$

As one can construct a 2-HST with constant distortion out of a star metric, the above lower bound complements that the algorithm from Corollary 3.9 is best-possible (up to a constant factor) for 2-HSTs under the regime of well-separated queries to the oracle.

D.3. Stronger Lower Bounds for Well-Separated Queries Model

Proof Sketch. Observe that, in the well-separated queries model, the adversary can modify predictions in order to mislead the algorithm. In particular, for the problem instance $\mathcal{T}_n$, the adversary can employ the same strategies as in Theorems 4.2 and 4.4, but over a longer duration, by concealing the server that is matched to the center request in the offline solution.

This concealment can be achieved by altering only a single server in each prediction. Consequently, the adversary can extend the execution of the strategy by additional k rounds for every error budget of 2.

□

We therefore obtain the following theorems:

Theorem D.2. *There exists an instance such that deterministic algorithm making well-separated queries with separation parameter k incurs cost of $2k - 1 + k\eta$, where $\eta = \sum_{t=1}^{\lfloor n/k \rfloor} \text{dist}(\text{OPT}_{tk}, P_{tk}) \leq 2(n/k - 1)$, while the minimum cost of a perfect matching with hindsight is 1.*

Theorem D.3. *There exists an instance such that randomized algorithm making well-separated queries with separation parameter k incurs cost of $\Omega(\log(k + k\eta))$, where $\eta = \sum_{t=1}^{\lfloor n/k \rfloor} \text{dist}(\text{OPT}_{tk}, P_{tk}) \leq 2(n/k - 1)$, while the minimum cost of a perfect matching with hindsight is 1.*

E. Other Deferred Proofs

Proof of Fact 2.2. The first half of the statement is easy to see since, in a minimum-cost perfect matching between $A \uplus C$ and $B \uplus C$, we can assume without loss of generality that every $v \in C$ is matched with v itself.

The second half of the statement is due to the triangle inequality. More precisely, let M_{AC} and M_{CB} be minimum-cost perfect matchings between A & C and between C & B , respectively. Now consider a perfect matching M_{AB} between A and B defined as follows: every $u \in A$ is matched with the counterpart $v \in B$ in M_{CB} of the counterpart $w \in C$ in M_{AC} of u , i.e., $(u, w) \in M_{AC}$ and $(w, v) \in M_{CB}$. Note that, by the triangle inequality, $d(u, v) \leq d(u, w) + d(w, v)$. We can thus derive

$$\text{dist}(A, B) \leq \text{cost}(M_{AB}) \leq \text{cost}(M_{AC}) + \text{cost}(M_{CB}) = \text{dist}(A, C) + \text{dist}(C, B).$$

□

Proof of Lemma 3.2. We claim that, for every round t ,

$$d(s_t, r_t) \leq \text{dist}(P_t, P_{t-1} \cup \{r_t\}) + \text{dist}(S_{t-1}, P_{t-1}) - \text{dist}(S_t, P_t).$$

Note that this claim immediately proves the lemma since the last two terms in the right-hand side telescope, and $\text{dist}(S_0, P_0) = \text{dist}(S_n, P_n) = 0$.

First, by the construction of p_t and s_t , we can derive

$$\begin{aligned} d(p_t, r_t) &= \text{dist}(P_t, P_{t-1} \cup \{r_t\}) - \text{dist}(P_t \setminus \{p_t\}, P_{t-1}) \\ &= \text{dist}(P_t, P_{t-1} \cup \{r_t\}) - \text{dist}(P_t, P_{t-1} \cup \{p_t\}); \end{aligned} \tag{6}$$

$$\begin{aligned} d(s_t, p_t) &= \text{dist}(S \setminus (S_{t-1} \setminus P_{t-1}), S \setminus (P_{t-1} \setminus S_{t-1})) - \text{dist}(S \setminus (S_{t-1} \setminus P_{t-1}) \setminus \{s_t\}, S \setminus (P_{t-1} \setminus S_{t-1}) \setminus \{p_t\}) \\ &= \text{dist}(S_{t-1}, P_{t-1}) - \text{dist}(S_t, P_{t-1} \cup \{p_t\}). \end{aligned} \tag{7}$$

We thus have

$$\begin{aligned} d(s_t, r_t) &\leq d(p_t, r_t) + d(s_t, p_t) \\ &= \text{dist}(P_t, P_{t-1} \cup \{r_t\}) - \text{dist}(P_t, P_{t-1} \cup \{p_t\}) + \text{dist}(S_{t-1}, P_{t-1}) - \text{dist}(S_t, P_{t-1} \cup \{p_t\}) \\ &\leq \text{dist}(P_t, P_{t-1} \cup \{r_t\}) + \text{dist}(S_{t-1}, P_{t-1}) - \text{dist}(S_t, P_t), \end{aligned}$$

where the first inequality is due to the triangle inequality, the equality comes from Equations (6) and (7), and the second inequality follows from the fact that $d(S_t, P_t) \leq d(S_t, P_{t-1} \cup \{p_t\}) + \text{dist}(P_{t-1} \cup \{p_t\}, P_t)$ due to Fact 2.2. This completes the proof of the claim. □

F. Generation of a Taxi Instance

A Taxi instance is generated using data from October 19, 2023 in the Chicago Taxi Trips (2013–2023) dataset (City of Chicago, 2024). The following fields are used in the instance construction:

- Trip Start Timestamp,

- Trip End Timestamp,
- Dropoff Centroid Longitude,
- Dropoff Centroid Latitude,
- Pickup Centroid Longitude,
- Pickup Centroid Latitude.

We partition the time span of October 19, 2023 into 5-minute intervals and uniformly sample one time point from this set. Given a sampled time point τ , we identify the latest 100 entries whose Trip End Timestamp does not exceed τ and treat them as servers. Similarly, we identify the earliest 100 entries whose Trip Start Timestamp is not earlier than τ and treat them as requests arriving sequentially. If fewer than 100 servers or 100 requests are available, we discard the instance and repeat the procedure with an independently sampled time point. The distance among servers and requests are defined as the Manhattan distance using the servers' (Dropoff Centroid Longitude, Dropoff Centroid Latitude) and the requests' (Pickup Centroid Longitude, Pickup Centroid Latitude).